Software Requirement Specifications

GeoSwap: A Proximity-Based Mobile Barter Radar System

Submitted by

Arifa S23BINFT1M01204

Submitted to

Mr. Wahaj Ali

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur

Meeting Details

Sr No	Details	Date	Supervisor Signature

Summary

This document outlines the technical and operational requirements for **GeoSwap**, a mobile application designed to modernize local bartering through real-time proximity tracking. While traditional e-commerce platforms focus on monetary sales and global shipping, **GeoSwap** targets the gap in local, item-for-item exchanges by using a "radar" interface to connect users within a specific geographic radius. Our research shows that while GPS discovery is popular in ride-hailing and social apps, it is rarely applied to the barter economy, leading to high "search costs" for people with underused goods. To solve this, the system integrates secure user registration, multimedia-rich listings, and a private negotiation chat, all built on a foundation of location accuracy and data security to ensure a reliable trading experience from discovery to completion.

Table of Contents

1. Introduction

GeoSwap is a location-aware mobile platform designed to modernize the traditional barter system by facilitating the direct exchange of goods and services through real-time proximity tracking. While modern e-commerce typically relies on currency-driven transactions and global shipping, this system focuses on the immediate, non-monetary value of local resources, using a specialized "Mobile Barter Radar" to connect users with trade opportunities in their immediate vicinity. By automating the discovery of nearby trade partners, the platform eliminates the high search costs usually associated with bartering, fostering a community-driven environment where users can repurpose surplus items for those they need. Ultimately, the system provides a seamless and sustainable alternative to traditional purchasing, encouraging environmental responsibility and social connectivity through secure, item-for-item swaps.

1.1. Purpose

The primary objective of **GeoSwap** is to establish a secure and highly intuitive location-aware platform that eliminates the necessity for monetary transactions in everyday resource acquisition. By automating the discovery of potential trade partners through a specialized radar interface, the project seeks to simplify the bartering experience and support a circular, waste-reducing economy. The goal is to provide a reliable digital medium where users can maximize the utility of their surplus belongings while fostering community-based sharing and social connectivity.

1.2. Scope

The scope of **GeoSwap** is focused on establishing a secure, location-aware ecosystem for non-monetary trade. The project boundaries are defined by the following technical and operational parameters:

Items to be Developed:

- **Mobile Application:** A cross-platform mobile interface (Android/iOS) for end-users to manage their accounts and trade activities.
- **Real-time Radar Module:** A dynamic, GPS-driven map interface that visualizes tradeable items based on the user's current geographic coordinates.

- **Item Management System:** A module allowing users to create detailed listings including descriptions, categories, and multimedia (photo) uploads.
- **Proximity Matching Algorithm:** A background service that identifies and notifies users of potential trade matches within a defined search radius.
- **In-App Negotiation Hub:** A secure, real-time chat interface for users to discuss item conditions and coordinate physical exchange details.
- **Administrative Portal:** A web-based dashboard for managing user accounts, verifying profiles, and moderating listing content to ensure community safety.

Items Not to be Developed:

- **Monetary Payment Gateway:** The system will not facilitate cash transactions, digital wallets, or credit card processing, as it is strictly a barter-only platform.
- **Logistics and Shipping Services:** The platform will not provide delivery or courier integration; users are responsible for the physical transfer of goods.
- **Legal Arbitration Framework:** The project does not include formal mediation services or legal support for disputes regarding item quality or trade outcomes.
- **E-commerce Cart System:** Unlike sales platforms, the system will not include a "shopping cart" or multi-item checkout functionality.

1.3. Product Perspective

GeoSwap is designed as a standalone mobile application that operates within a distributed system environment, utilizing cloud-hosted services and global positioning systems. The system integrates multiple architectural layers to ensure that the **Mobile Barter Radar** module functions as a seamless extension of the user's physical surroundings.

The developed module fits into the broader system architecture through the following integrations:

- **GPS and Location Services:** The system interfaces with external Global Positioning System (GPS) providers to capture real-time coordinate data. This data is the primary input for the Radar module, allowing it to dynamically filter and display available items relative to the user's current location.

- **Centralized Cloud Database:** The application communicates with a centralized database (MySQL) to synchronize item listings, user statuses, and proximity data. This ensures that when a user lists a new item, it becomes instantly visible on the "radar" of other users within the specified radius.
- **Real-time Communication Layer:** The module integrates with an external messaging service (Firebase) to facilitate the Negotiation Hub. This allows the discovery phase on the radar to transition smoothly into active trade discussions without leaving the application environment.
- **Administrative Synchronization:** The mobile frontend is linked to a web-based Administrative Portal, ensuring that all local discovery and trading activities remain compliant with community standards through real-time moderation and user verification protocols.

By operating at the intersection of mobile hardware and cloud-based logic, **GeoSwap** transforms a standard smartphone into a localized discovery tool, effectively mapping the physical barter economy onto a digital interface.

1.4. User Characteristics

The **GeoSwap** ecosystem involves several distinct human roles, each with specific responsibilities and levels of technical expertise. The following roles interact with the system to ensure a smooth and secure bartering experience:

- **General Users (Customers):** These are the primary actors of the system, including students and local community members. They use the mobile application to list surplus items, browse the "Barter Radar," and initiate trade requests. These users are expected to have basic smartphone proficiency and a general understanding of map-based navigation.
- **System Administrator:** The Admin possesses full access to the web-based administrative portal. Their role is technical and supervisory, involving the management of system configurations, oversight of the user database, and final decision-making on reported content or disputed accounts. The Admin ensures the overall health and security of the platform.

- **Data Entry Operators / Moderators:** These users act as intermediaries between the general public and the system's database. Their primary tasks include verifying the accuracy of new user registrations (checking ID details and phone numbers) and moderating item listings to ensure they comply with community standards. They possess moderate technical skills and focus on maintaining data integrity and platform safety.
- **Guest Users:** Unregistered individuals who download the app may interact with the system in a limited capacity. They can view a restricted version of the barter radar to see general activity in their area but must register and undergo verification to view specific item details or initiate a swap.

1.5. Similar apps and systems/Literature Review

Existing platforms for local trade and discovery were analyzed to identify functional gaps in the current barter economy. The following table compares popular systems with the proposed **GeoSwap** solution:

System / App	Key Features	Shortcomings & Gaps
OLX / Facebook Marketplace	Large-scale local classifieds; category-based browsing; user messaging.	Primarily optimized for monetary sales; lacks a dedicated barter-only logic and real-time "radar" discovery.
Freecycle / Buy Nothing	Focuses on gifting items for free to reduce waste; community-based groups.	Relies on outdated list-based or forum-style interfaces; slow to navigate with no real-time GPS tracking.
Tinder / Social Apps	Uses real-time GPS "radar" discovery to show nearby profiles.	Designed exclusively for social networking; lacks inventory management and item-matching functionality.
Listia / Bunz	Dedicated swap platforms using virtual credits or points.	Requires "points" or "virtual currency," which complicates the pure barter process; lacks a dynamic map-based radar.

Gap Analysis: The comparison highlights a significant "Discovery Gap" in the barter economy. While social apps have mastered real-time proximity (the "Radar" concept) and e-commerce apps have mastered inventory, no single platform effectively combines both for a pure, non-monetary exchange. **GeoSwap** fills this void by integrating a real-time GPS radar with a dedicated barter logic, significantly reducing the "search cost" for local users.

1.6. Proposed Technologies

To ensure high performance, real-time synchronization, and robust data management, **GeoSwap** will be developed using a specialized multi-tier architecture. The selected technology stack is designed to handle complex geospatial data while maintaining a smooth user experience.

- **Front-end: Flutter (Cross-platform Mobile UI)**

  The primary interface will be built using **Flutter**, Google's UI toolkit. This allows for the development of a natively compiled, high-performance application for both **Android and iOS** from a single codebase. It is particularly suited for this project due to its ability to render smooth, custom animations required for the "Barter Radar" interface and interactive maps.

- **Middle-end: Python (Geospatial Logic & Loop Matching Algorithm)**

  A dedicated **Python** layer will handle the core intelligence of the system. This "Middle-end" will manage the complex **Geospatial Logic** required to calculate distances between users in real-time. Furthermore, Python will power the **Loop Matching Algorithm**, which identifies multi-user trade opportunities (e.g., User A has what User B wants, and User B has what User A wants) to increase the probability of successful swaps.

- **Back-end: PHP (RESTful API)**

  The server-side communication will be managed using **PHP**. This layer will function as a **RESTful API**, serving as the secure bridge between the mobile application and the database. PHP's efficiency in handling server-side requests ensures that user authentication, item uploads, and status updates are processed reliably and quickly.

- **Database: MySQL (User Data & Inventory Management)**

  MySQL will serve as the central relational database management system. It will be responsible for storing all persistent data, including:

  - **User Profiles:** Secure storage of credentials and verification statuses.
  - **Inventory Management:** Categorized listings of all items available for trade, including their metadata and geographic coordinates.
  - **Transaction History:** Records of completed swaps and active negotiations.

2. Requirements

The functional requirements of the **GeoSwap** system center on establishing a seamless, location-aware environment for item-to-item exchange. The core functionality begins with a robust user registration and verification process, where the system validates profiles to ensure a secure community environment. Once verified, users can utilize the item management module to list surplus goods by uploading detailed descriptions, categories, and photographs. The most critical function is the dynamic "Barter Radar," which captures the user's real-time GPS coordinates and visualizes nearby tradeable items on an interactive map. To facilitate successful swaps, the system implements a proximity-matching algorithm that notifies users when relevant items enter their search radius. Once a potential match is identified, the integrated negotiation hub provides a real-time chat environment for users to discuss item conditions and finalize the logistics of the physical exchange. On the administrative side, the system includes comprehensive moderation tools to monitor listings, manage user reports, and maintain the integrity of the decentralized marketplace, ensuring that all activities align with the platform's community standards.

2.1. Function Requirements

2.1.1. Register Account

- **Name:** FR001
- **Purpose:** The registration process is required for an individual to become a verified member of the system and access the bartering community.
- **User(s):** General User, Data Entry Operator

- **Input:**
    - *Name:* Actual name of user according to ID card.
    - *Password:* Must be 8 characters long and must include numbers.
    - *Address:* Actual residence address for local trade verification.
    - *Phone:* Working phone number; must be on WhatsApp.
    - *Display Picture:* Latest clear face picture.
    - *Approval from Super User:* The Admin or Data Entry Operator will verify the information and approve the status of the user as a member of the system.
- **Output:** User will be the member of system and able to login

2.1.2. Item Listing Management

- **Name:** FR002
- **Purpose:** To allow users to upload and manage the items they wish to barter within the community.
- **User(s):** General User
- **Input:**
    - *Category*: Selection from a predefined list (e.g., Electronics, Books, Fashion).
    - *Condition*: A clear status of the item (e.g., New, Gently Used, Refurbished).
    - *Item Description*: Detailed text explaining features or flaws.
    - *Images*: At least two high-quality photos showing the item from different angles.
    - *Location*: GPS-linked location where the item is available for trade.
- **Output:** The item is successfully listed in the database and visible on the Barter Radar.

2.1.3. Real-Time Radar Discovery

- **Name:** FR003
- **Purpose:** To visualize available trade items within a specific geographic radius based on the user's current GPS location.
- **User(s):** Guest and Verified Users
- **Input:**
    - *GPS Coordinates:* Real-time latitude and longitude data from the user’s mobile device.
    - *Search Radius:* User-selected distance for item filtering.

- **Output:** An interactive map interface displaying icons for all tradeable items within the selected proximity.

2.1.4. In-App Negotiation Hub

- **Name:** FR004
- **Purpose:** To provide a secure real-time communication channel for users to coordinate swap details.
- **User(s):** General User
- **Input:**
    - *Message Content:* Text-based inquiries regarding item condition or exchange location.
    - *Swap Request:* A formal trigger to initiate or accept a specific trade agreement.
- **Output:** A secure, real-time chat log and an updated status of the trade negotiation.

2.1.5. User Verification & Moderation

- **Name:** FR005
- **Purpose:** To maintain platform safety by managing user reports and verifying the authenticity of participants.
- **User(s):** Admin, Data Entry Operator
- **Input:**
    - *Report Log:* Content or profiles flagged by the community for review.
    - *Verification Documents:* Digital copies of ID credentials submitted during registration.
- **Output:** Actionable results such as account approval, suspension, or content removal.

2.1.6. Proximity Matching Notification

- **Name:** FR006
- **Purpose:** To automatically alert users when an item they desire becomes available within their immediate vicinity.
- **User(s):** General User
- **Input:**
    - *Watchlist Keywords:* Specific item types or categories the user is tracking.

    - *Geospatial Proximity:* Distance calculation between the user and a newly listed matching item.
- **Output:** A push notification sent to the mobile device when a match is detected within the defined range.

2.2. Non-Functional Requirements

2.2.1. Performance

The system must be highly responsive to ensure a smooth user experience, especially during map interactions. The **Mobile Barter Radar** should update item markers within 2 seconds of a location change or filter adjustment. Additionally, the backend must be optimized to support at least 500 concurrent users without significant latency in the negotiation chat or item search results.

2.2.2. Security and Privacy

Since the platform handles real-time GPS data and personal identification (ID cards), security is paramount. All sensitive user data, including passwords and residential addresses, must be encrypted using industry-standard protocols (e.g., AES-256). Furthermore, precise GPS coordinates of a user's home should never be displayed directly; instead, a "fuzzy" location or generalized radius should be used on the public radar to protect user privacy.

2.2.3. Availability and Reliability

GeoSwap should maintain an uptime of 99.5% to ensure users can access their trade agreements and the radar at any time. The system must implement robust error-handling mechanisms to prevent crashes during GPS signal loss. In the event of a server failure, the system should be able to recover and restore the most recent state of all active negotiations from the MySQL database.

2.2.4. Usability

The mobile interface must be intuitive, requiring minimal training for the average smartphone user. Navigation between the radar, the item listing page, and the chat hub should be accessible within a maximum of two taps. The UI design will follow minimalist principles to ensure clarity, even when the map is populated with multiple item markers.

2.2.5. Scalability

The architecture must be designed to scale horizontally. As the user base grows from a single university campus to multiple cities, the Python-based middle-end and PHP backend should be capable of handling increased data loads by integrating cloud-based load balancers and database indexing.

2.2.6. Compatibility

The application developed in **Flutter** must be compatible with **Android (version 8.0 and above)** and **iOS (version 12.0 and above)**. It must also be responsive to various screen sizes and resolutions, ensuring that the Barter Radar remains functional on both budget smartphones and flagship devices.

3. Use Cases and Flow of Processes

This section provides the formal representation of the process flows for **GeoSwap**. It begins with a system-level overview and then provides detailed flows and diagrams for each core use case.

Actor 1: General User (Customer)

Actor 2: Data Entry Operator (DEO)

Actor 3: System Administrator

Actor 4: External GPS Service

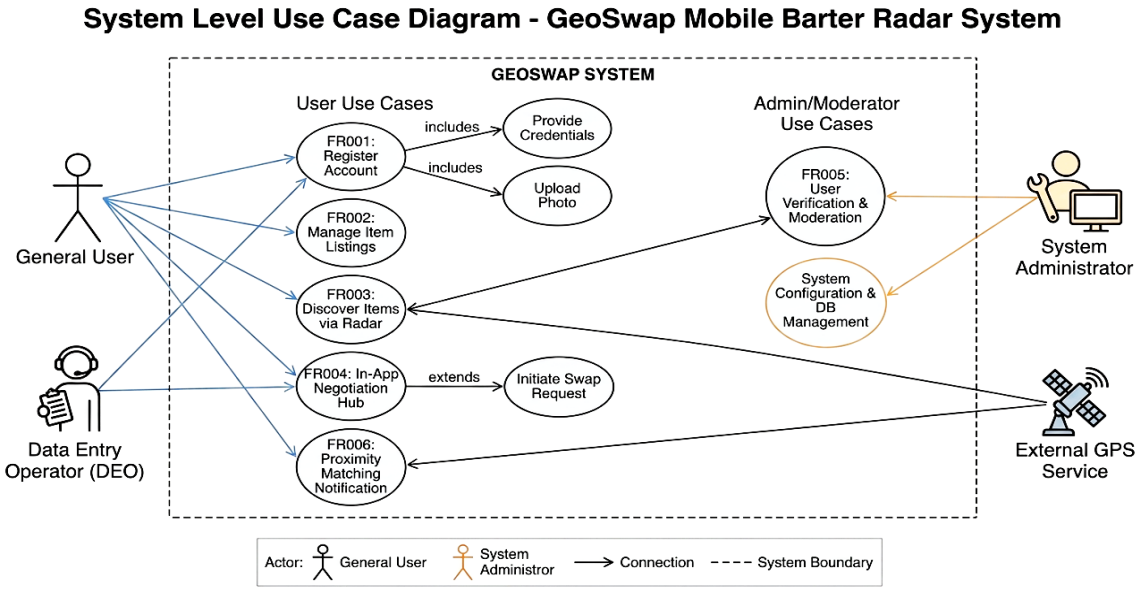

Figure 1: System Level Use Case Diagram

3.1. Use Case 1

ID	UC001
Name	Register Account
Description	This use case describes the process by which a new user submits their details for verification and how the system manages the pending state before approval.
Requirement(s)	FR001
Actor(s)	General User, Data Entry Operator (DEO), Admin
Precondition	The user has the GeoSwap mobile app installed and does not possess an existing account associated with their phone number.
Postcondition	A user profile is stored in the database with a "Pending" status, and a notification is queued for the DEO.
Basic Flow	***Basic Flow*** 1. The General User selects the "Sign Up" option on the mobile interface 2. The General User enters their Name, WhatsApp-active Phone Number, Password, and Address, and uploads a Display Picture. 3. The System performs real-time validation: 3.1.1. Verifies phone number is exactly 11 digits. 3.1.2. Verifies password is >= 8 characters and includes at least one digit. 3.1.3. Validates image format (JPG/PNG). 4. Upon successful validation, the System encrypts the password and creates a profile with **Status: Pending.** 5. The System logs the registration timestamp and notifies the DEO via the Admin Portal dashboard ***Alternative Flow*** 1. The Admin authenticates into the Web Portal. 2. The Admin navigates to the "User Management" section and selects "Add New User."

	3. The Admin enters the user data directly. 4. The System bypasses the "Pending" state and creates an **Active** account immediately. ***Exceptions*** 1. **Validation Failure:** If inputs are malformed, the System highlights the specific field and disables the "Submit" button until corrected. 2. **Duplicate Account:** If the phone number already exists in the database, the System displays a "Number already registered" message. 3. **Database Timeout:** If the backend connection fails, the System displays a "Server busy, please retry later" message and retains the user's input locally for one retry attempt.

Diagram:

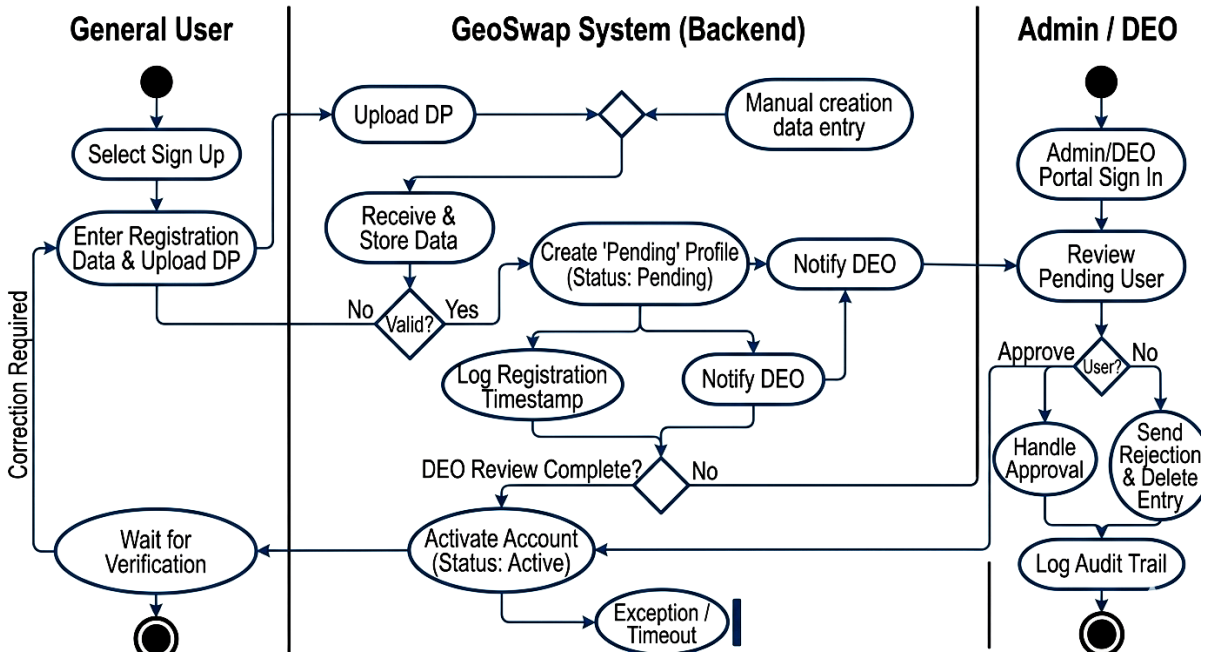

Figure 2: Activity Diagram for Register Account (UC001)

3.2. Use Case 2

ID	UC002
Name	Item Listing Management
Description	Describes the process of a verified user digitizing their surplus goods by uploading detailed descriptions and photographs to make them discoverable within the decentralized community.
Requirement(s)	FR002, FR006
Actor(s)	Verified User
Precondition	The user must be logged in and their account status must be "Verified" following approval from an Admin or Data Entry Operator.
Postcondition	The item is stored with **Status: Available**; indexed for Geo-spatial querying.
Basic Flow	***Basic Flow*** 1. The User selects "Add Item" on the dashboard. 2. The User provides Title, Description, Category, and Condition. 3. The User uploads a minimum of two images. 4. The System retrieves the device's GPS coordinates or allows the user to place a manual map pin. 5. The System validates inputs (checks for empty fields, image count, and coordinate accuracy). 6. The System saves the record to the MySQL database, linking the `item_id` to the `user_id`. 7. The System triggers the **Proximity Matching Algorithm** to identify potential barter matches. 8. A notification is broadcast to matching users. ***Alternative Flow*** 1. (Internal System Action): The proximity matching algorithm (FR006) detects the new listing and triggers a background

	broadcast to users within a 10km radius who have matching items in their "Watchlist." ***Exceptions*** 1. **Validation Fail:** System highlights specific errors (e.g., "Description must be > 20 characters"). 2. **Location Permission Denied:** If GPS is off, the system forces a manual "Map Pin" selection. The system must not allow submission without coordinates. 3. **Media Optimization Fail:** If the image file size is too large, the system automatically compresses the image before transmission to the server.

Diagram:

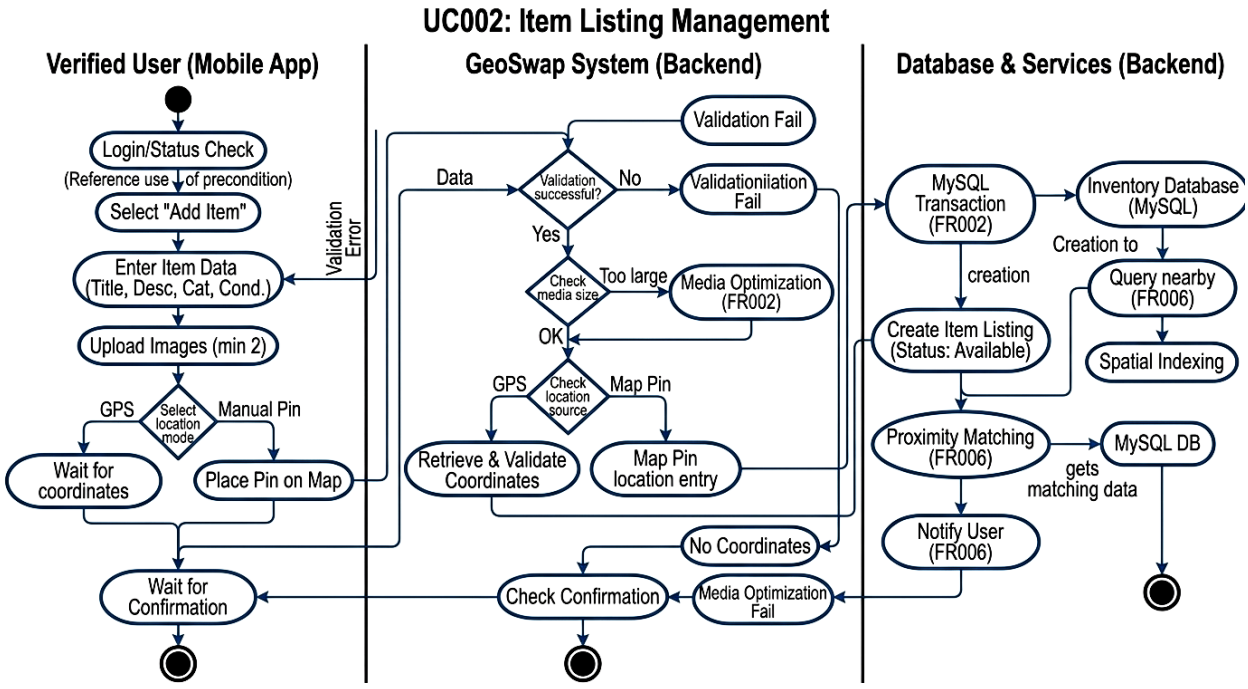

Figure 3: Activity Diagram for Item Listing Management (UC002)

3.3. Use Case 3

ID	UC003
Name	Real-Time Radar Discovery
Description	Describes how a user visualizes available tradeable items in their immediate geographic vicinity using an interactive map interface powered by live GPS data.
Requirement(s)	FR003
Actor(s)	Verified User, Guest User (Restricted), External GPS Service.
Precondition	The user must be logged in and the mobile device must have location services enabled.
Postcondition	An interactive map is displayed with markers representing items available for barter within the user's defined proximity.
Basic Flow	***Basic Flow*** 1. The User selects the "Radar" tab from the navigation bar. 2. The System requests current coordinates from the External GPS Service. 3. The System retrieves the User's preferred search radius (default: 5km). 4. The system executes a geospatial scan using **Haversine mathematical logic** to ensure compatibility across database environments. The query must be optimized to execute in < 2 seconds to meet performance requirements in NFR 2.2.1. 5. The System renders the "Radar View," populating the map with interactive item markers. The System applies a privacy offset (fuzzing logic) to item coordinates before rendering markers on the map. 6. The User taps a marker to view a brief summary (Title, Distance, and Thumbnail). ***Alternative Flow***

	1. **Radius Adjustment:** The User adjusts the distance slider; the System clears markers and re-executes the geospatial query. 2. **Manual Location Entry:** The User inputs a specific city or uses a map-picker to set a focal point other than their current GPS location. ***Exceptions*** 1. **GPS Signal Lost:** If coordinates are unavailable, the System prompts the User to check settings or switch to "Manual Location Entry." 2. **No Items Found:** If the query returns zero results, the System suggests: "No items nearby. Try expanding your search radius." 3. **Connectivity Error:** If the backend is unreachable, the System displays a "Network Error" and provides a "Retry" option.

Diagram:

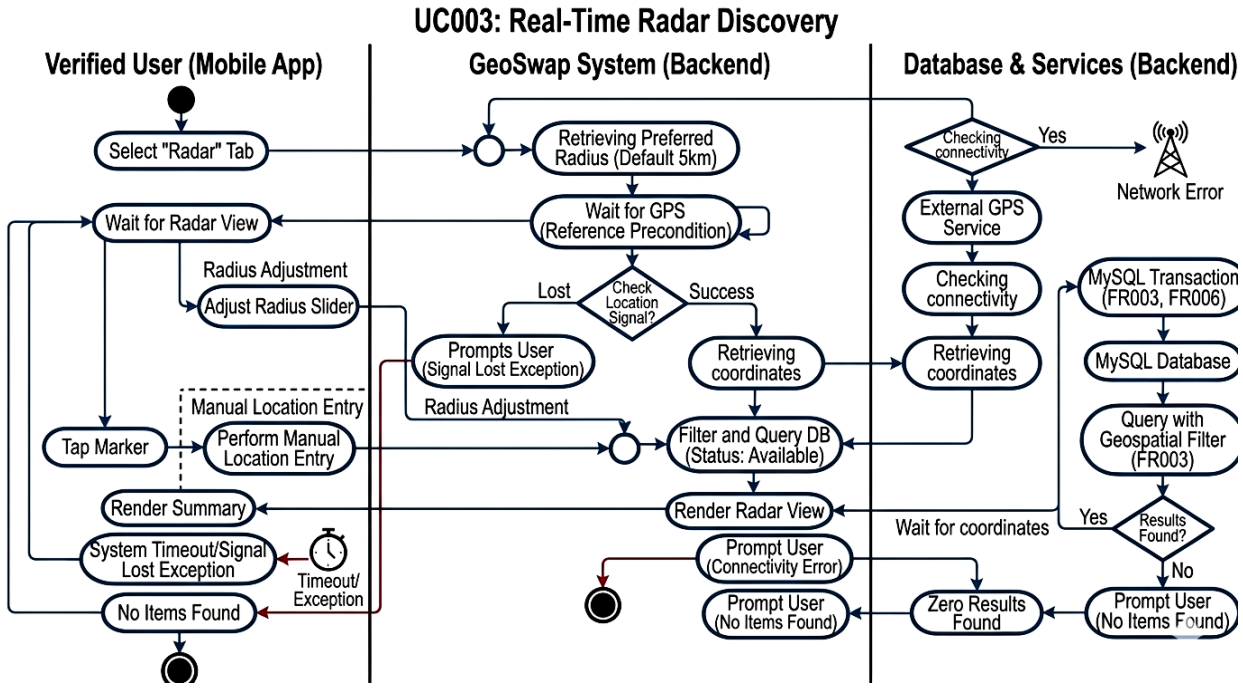

Figure 4: Activity Diagram for Real-Time Radar Discovery (UC003)

3.4. Use Case 4

ID	UC004
Name	In-App Negotiation Hub
Description	Describes the secure real-time communication process between two users to discuss item specifics, exchange additional media, and finalize the logistics of a physical barter trade.
Requirement(s)	FR004
Actor(s)	Verified User
Precondition	The user A (Initiator) must have identified a desired item on the Radar belonging to User B (Receiver), and both users must be verified members.
Postcondition	A formal swap agreement is recorded, and the item status is updated to reflect the ongoing trade.
Basic Flow	***Basic Flow*** 1. User A selects "Chat / Propose Swap" from an item's detail view. 2. The System opens a dedicated chat interface and sends a real-time notification to User B. 3. Both Users exchange text messages regarding item condition and meeting points. 4. User A selects a specific item from their own inventory to offer in exchange. 5. User A triggers a formal "Swap Request" (FR004.1). 6. User B receives the request and selects "Accept Swap." 7. The System updates the status of both items to "Reserved" and records the transaction in the MySQL database. 8. The System suggests alternative 3-way trades using the Loop Matching Algorithm if a direct swap is declined. ***Alternative Flow*** 1. **Counter-Offer:** User B declines the initial request but suggests a different item from User A's inventory.

	2. **Declining Swap:** User B selects "Decline." The System notifies User A and closes the formal request while keeping the chat open for further discussion. ***Exceptions*** 1. **Item No Longer Available:** If the item is swapped with another user during negotiation, the System disables the "Send Request" button and notifies User A. 2. **Network Interruption:** If a message fails to send due to poor connectivity, the System displays a "Retry" icon next to the message bubble.

Diagram:

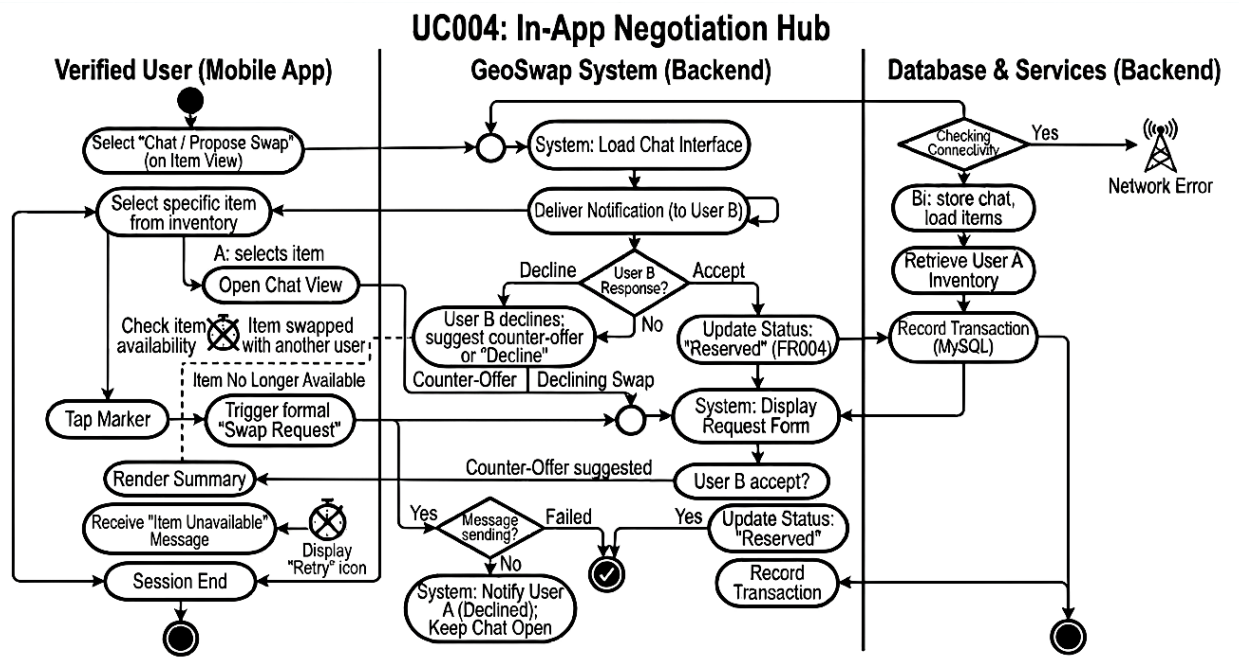

Figure 5: Activity Diagram for In-App Negotiation Hub (UC004)

3.5. Use Case 5

ID	UC005
Name	User Verification & Moderation
Description	The administrative process of auditing registration documents and managing community reports to ensure the integrity and safety of the platform.
Requirement(s)	FR005
Actor(s)	Admin, Data Entry Operator (DEO)
Precondition	A new user has submitted registration data or a listing has been flagged for review.
Postcondition	The user's status is updated (e.g., Verified, Suspended) and appropriate content actions are logged.
Basic Flow	***Basic Flow*** 1. **Login:** The Actor logs into the GeoSwap Administrative Web Portal. 2. **Authentication:** The System validates credentials against the database (**FR005**). 3. **Queue Access:** The Actor accesses the "Verification/Moderation Queue." 4. **Data Retrieval:** The System retrieves pending registration records or flagged item details. 5. **Review:** The Actor reviews the case details (ID documents or reported content). 6. **Decision:** The Actor selects an action: **Approve**, **Reject**, or **Delete Listing**. 7. **Database Update:** The System updates the record status and logs the deletion if applicable (**FR005**). 8. **Audit Trail:** The System records the Actor ID and timestamp in the Audit Log (**FR005**).

	9. **Confirmation:** The System displays a "Status Updated" confirmation to the Actor. ***Exceptions*** 1. **Insufficient Evidence: I**f the submitted documents are unclear, the Actor selects Request Re-upload. The System notifies the user to submit new data, and the record remains "Pending." 2. **Session Timeout:** If the Actor is idle, the System terminates the session and requires a re-login to protect administrative data. 3. **Update Failure:** If the database fails to commit the change, the System displays an error and prompts the Actor to retry.

3.6. Use Case 6

ID	UC006
Name	Proximity Matching Notification
Description	An automated background service that dynamically matches new listings with user "Watchlists" and triggers alerts based on geospatial proximity.
Requirement(s)	FR006
Actor(s)	System (Notification Engine), Verified User
Precondition	The user has a "Verified" status, has background location enabled, and has at least one active keyword/category in their Watchlist.
Postcondition	A push notification is delivered, and the match is recorded in the **Match History** for the user. (This allows them to find the item even if they accidentally swipe the notification away).
Basic Flow	***Basic Flow*** 1. The System is triggered by a successful item upload in UC002. 2. The System executes a geospatial scan using Haversine logic to identify users within a 10km radius (or user-defined radius) of the item's coordinates.

	3. The System performs a keyword intersection between the new item's metadata and the identified users' Watchlists. 4. For every match found, the System formats a notification payload containing the Item Title and Distance. 5. The System dispatches a push notification via Firebase Cloud Messaging (FCM) or a similar service. 6. The User interacts with the notification to launch the app directly into the Item Detail view. ***Exceptions*** 1. **Location Latency:** If the user's last known location is older than 30 minutes, the System ignores the match to maintain accuracy. 2. **Notification Throttling:** If the user has already received 5+ notifications in an hour, the system batches the alerts to avoid spamming.

Diagram:

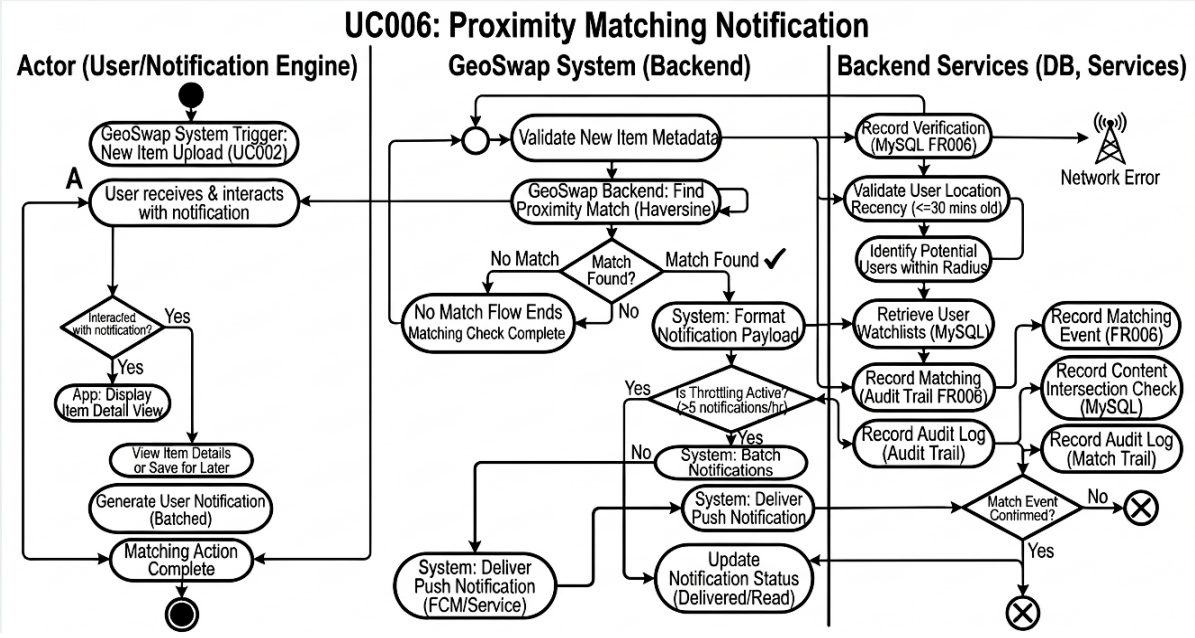

Figure 6: Activity Diagram for Proximity Matching Notification (UC006)

4. References

[1] R. W. Sinnott, "Virtues of the Haversine," *Sky and Telescope*, vol. 68, no. 2, p. 158, Aug. 1984.

[2] Google, "Firebase Cloud Messaging (FCM): Build and send cross-platform messages," *Firebase Documentation*, 2026. [Online]. Available: https://firebase.google.com/docs/cloud-messaging. [Accessed: May 1, 2026].

[3] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2003.